Data Cleaning and Preparation

Slides for *MATH 251 – Database Queries*

Ali Tavasoli

Based on the slides and book by Antonio Badia
edited and revised by Stefanie Scherzinger & Ali Tavasoli

JAMES MADISON UNIVERSITY

Outline

1. Data Cleaning
   - Missing Data
   - Outlier Detection
   - Duplicate detection and removal

Data Cleaning

- **Data cleaning** (also called **data wrangling**, **data munging**):
  - The process of finding and fixing problems in a dataset.
- Data cleaning can be **complex and messy** because datasets may have many different kinds of problems.
- Common issues include:
  - **Proper data:** values should have **correct type** and **proper format**.
  - **Missing data:** some values are not recorded.
  - **Outliers:** values that are very different from most of the other values.
  - **Duplicate data:** multiple records refer to the same real-world entity.
  - **Noisy data**: data with **irrelevant values** that can distort analysis

Data Cleaning (Cont.)

- **Data cleaning** also prepares data for analysis using **attribute transformations**.
- These transformations change values into a **consistent format, pattern, or range**.
- For **categorical attributes**, a common step is **standardization** (sometimes called normalization): assigning a **consistent representation** to each category.
- For **numerical attributes**, common transformations include **scaling** and **normalization**.
- **Temporal data** (dates, times) often require **proper formatting**.

Data Cleaning: Numbers

- **Scaling** ensures values lie within a **fixed range**.
- A common method is **linear scaling** (min-max scaling), which maps values to **[0,1]**.
- Each value v is transformed as:

$$\frac{v - \min}{\max - \min}$$

- In SQL: use UPDATE-SET to apply the transformation

```
UPDATE Dataset
SET Attr = (Attr - (SELECT MIN(Attr) FROM Dataset)) /
           ((SELECT MAX(Attr) FROM Dataset) -
            (SELECT MIN(Attr) FROM Dataset));
```

Try it: https://www.db-fiddle.com/f/osaHfDqPeV249h1itJC4yd/0

Data Cleaning: Numbers

- **Logistic scaling** maps values to **(0,1)**.

$$\frac{1}{1+e^{-v}}$$

```
UPDATE Data
SET Attr = 1 / (1 + EXP(-Attr));
```

- **Z-score** measures how far a value is from the mean in units of standard deviation.

$$\frac{v-\mu}{\sigma}$$

```
UPDATE Data
SET Attr = (Attr -
            (SELECT AVG(Attr) FROM Data)) /
           (SELECT STD(Attr) FROM Data);
```

- **Other transformations:** $\log(v)$, $1/v$, $\sqrt{v}$, v^n, etc.

Data Cleaning: Strings

- **Cleaning categorical data** means ensuring each category has **one consistent name**.
- For example: `Germany`, `germany`, `GERMANY` should all be treated as **the same category**.
- In practice, we define a **canonical form** (standard representation).

Student-id	...	Department
1	...	Dept of Economics
2	...	Econ
3	...	Department of Econ

Example (standardizing to "Economics"):

```
UPDATE Student
SET Department = 'Economics'
WHERE position('Econ' IN Department) > 0;
```

[1]

[1] **position(substring IN string)** returns the starting index of a substring.

SQL: position()

- **position(substring IN string)** returns the **starting index** of a substring.
- If found → **number** ≥ 1 — if not found → **0**
- Examples:

```
position('Econ' IN 'Dept of Economics') → 9
position('Econ' IN 'Econ')              → 1
position('Econ' IN 'Mathematics')       → 0
```

- **Case-sensitive:**

```
position('Econ' IN 'economics') → 0
```

- Safer usage:

```
WHERE position('econ' IN LOWER(Department)) > 0
```

- **Interpretation:** `position(...) > 0` ⇒ **"contains substring"**

Data Cleaning: Strings (Cont.)

- **Categorical inconsistency:** the same category may appear in different forms.
- Example: `F`, `female`, `femme` ⇒ **Female**
- Use `CASE` to map values to a **canonical form**:

```
CASE WHEN gender = 'F' THEN 'Female'
     WHEN gender = 'female' THEN 'Female'
     WHEN gender = 'femme' THEN 'Female'
     ELSE gender
     END AS gender_cleaned
```

Try it: `https://www.db-fiddle.com/f/7RTKQ4mjtDoQTsjNaiXFRp/0`

Source: SQL for Data Analysis (O'Reilly)

Data Cleaning: Strings (Cont.)

Data cleaning is also used to to add meaning to raw numeric data, e.g., by creating a new variable

- **Example:** survey data with likelihood of recommending a company.
- **Goal:** create new categories (**enrichment**) from existing data.
- Using **ranges**:

```
SELECT response_id, likelihood,
  CASE WHEN likelihood <= 6 THEN 'Detractor'
       WHEN likelihood <= 8 THEN 'Passive'
       ELSE 'Promoter'
       END AS response_type
FROM nps_responses;
```

- Using **explicit sets (IN)**:

```
CASE WHEN likelihood IN (0,1,2,3,4,5,6) THEN 'Detractor'
     WHEN likelihood IN (7,8) THEN 'Passive'
     WHEN likelihood IN (9,10) THEN 'Promoter'
     END AS response_type
```

- **Idea:** map numeric values to **meaningful categories**.

Data Cleaning: Strings (Cont.)

Alternatives to CASE

- **CASE** works well for **small lists** of variations.
- For **larger or evolving lists**, use a **lookup table**.
- Then use a **JOIN** to retrieve cleaned values.
- **Practice:** start with CASE, switch to lookup when it grows.

Lookup table

gender_cleaning

original	cleaned
F	Female
female	Female
femme	Female
Male	Male

Using JOIN

```
SELECT id, name, gender, birthdate,
       cleaned AS gender_cleaned
FROM persons p
JOIN gender_cleaning c
  ON p.gender = c.original;
```

Try it: https://www.db-fiddle.com/f/n1wyY5isbcyUU5TiNTFhMr/0

Data Cleaning: Strings (Cont.)

String functions (vary by system):

- **Cleaning** (modify characters):
  - `TRIM()`, `LOWER()`, `UPPER()`
- **Padding** (add characters):
  - `LPAD()`, `RPAD()`
- **Finding** (locate substrings):
  - `POSITION()`, `STRPOS()`
- **Extract / Split**:
  - `SUBSTR()`, `SPLIT()`
- **Concatenation** (combine strings):
  - `CONCAT()`, `||`
- **Replacement** (modify parts):
  - replace or substitute substrings

String Functions in Action

Example of messy user data: `'  SARAH_johnson123@GMAIL.COM '`

- **Cleaning**: `LOWER(TRIM(name))`
  ⇒ 'sarah_johnson123@gmail.com'
- **Finding**: `POSITION('@' IN name)`
  ⇒ position of '@'
- **Extract**: `SUBSTR(name, 1, POSITION('@' IN name)-1)`
  ⇒ 'sarah_johnson123'
- **Concatenate**: `CONCAT(first_name, ' ', last_name)`
  ⇒ 'Sarah Johnson'
- **Replacement**: `REPLACE(name, '_', ' ')`
  ⇒ 'sarah johnson123'

Data Cleaning: Strings (Example)

Example: IMDB data `'Amarcord (1973)'`

Extract year (with parentheses)
⇒ '(1973)'

```
SELECT RIGHT(title, 6)
FROM imdb;
```

Extract title
⇒ 'Amarcord'

```
SELECT LEFT(title,
  STRPOS(title,'(') - 2)
FROM imdb;
```

Extract year (clean)
⇒ '1973'

```
SELECT SUBSTR(title, 11, 4)
FROM imdb;
```

Reformat
⇒ 'Amarcord - 1973'

```
SELECT CONCAT(
  LEFT(title,
    STRPOS(title,'(') - 1),
  ' - ',
  SUBSTR(title, 11, 4)
)
FROM imdb;
```

Data Cleaning: Dates

- **Some of the most problematic types of data in SQL:** Dates come in many formats ⇒ prone to errors in sorting and filtering
- **DATE** (standard format)

  `YYYY-MM-DD`

  ⇒ sorts correctly in chronological order
- **TIME** (standard format)

  `HH:MM:SS[.fraction]`
- **DATETIME (timestamp)** (standard format)

  `YYYY-MM-DD HH:MM:SS`
- **INTERVALS** (standard formats)
  - Year–Month: `YYYY-MM`
  - Day–Time: `DD HH:MM:SS.F`

Data Cleaning: Dates (Arithmetic)

Time arithmetic in SQL

Add days
⇒ '2001-10-05'

```
DATE '2001-09-28' + 7
-- '2001-10-05'
```

Add hours (interval)
⇒ '2001-09-28 01:00:00'

```
DATE '2001-09-28'
+ INTERVAL '1 hour'
-- '2001-09-28 01:00:00'
```

Add time
⇒ '2001-09-28 03:00:00'

```
DATE '2001-09-28'
+ TIME '03:00'
-- '2001-09-28 03:00:00'
```

Combine intervals
⇒ '1 day 01:00:00'

```
INTERVAL '1 day'
+ INTERVAL '1 hour'
-- '1 day 01:00:00'
```

Note: – works similarly for subtraction

Data Cleaning: Dates – Postgres

- **Flexible input formats**
  Postgres accepts multiple date formats: `'January 8, 2017'`, `'2017-Jan-08'`, `'08-Jan-2017'`, `'1/8/2017'`
  Note: last format is **ambiguous**

- **Extracting components**
  `EXTRACT()` retrieves parts of a date/time:

```
SELECT cleaned_date,
  EXTRACT('year'   FROM cleaned_date) AS year,
  EXTRACT('month'  FROM cleaned_date) AS month,
  EXTRACT('day'    FROM cleaned_date) AS day,
  EXTRACT('hour'   FROM cleaned_date) AS hour,
  EXTRACT('minute' FROM cleaned_date) AS minute,
  EXTRACT('second' FROM cleaned_date) AS second,
  EXTRACT('decade' FROM cleaned_date) AS decade,
  EXTRACT('dow'    FROM cleaned_date) AS day_of_week
FROM Data;
```

Data Cleaning: Dates – Postgres

Useful time functions

- **Current values**: `current_date()`, `current_time()`, `now()`
- **Create DATE**: `make_date(year, month, day)`
- **Create TIME**: `make_time(hour, min, sec)`
- **Create TIMESTAMP**: `make_timestamp(year, month, day, hour, min, sec)`
- **Create INTERVAL**: `make_interval(years, months, weeks, days, hours, mins, secs)`
- **Create an interval**:

  ```
  SELECT make_interval(days => 2, hours => 3);
  ```

  ⇒ '2 days 03:00:00'

Data Cleaning: Dates – Example in a Database

Example: table `events`

id	event_name	event_time
1	Exam 1	2026-02-10 09:30:00
2	Review	2026-02-08 14:00:00
3	Project	2026-02-20 16:45:00

- **Create a new timestamp**

```
SELECT make_timestamp(2026, 2, 15, 10, 30, 0);
```

$\Rightarrow$ '2026-02-15 10:30:00'

- **Shift time by an interval**

```
SELECT event_name,
       event_time,
       event_time + make_interval(days => 7) AS next_week
FROM events;
```

$\Rightarrow$ moves each event 7 days later

Data Cleaning: Dates – Postgres

- Casting can be used to create times from strings.
- Assume the data contains a string-based attribute `thedate` with values like '15-04-2015' and we want to convert them into dates:

```
SELECT (SUBSTR(thedate, 7, 4) || '-' || SUBSTR(thedate, 4, 2)
        || '-' || LEFT(thedate, 2))::DATE AS date
  FROM Data;
```

- Also, functions `TO_DATE(text, pattern)` and `TO_TIMESTAMP(text, pattern)` can be used for conversion.
- The pattern argument indicates how the conversion should interpret the string; `TO_DATE('05 Dec 2000', 'DD Mon YYYY')` indicates that '05' is the day, 'Dec' the value of the month, and '2000' the value of the year.

dbfiddle: `https://www.db-fiddle.com/f/pL1zud2T1SRNkUBDXN12KV/0`

Data Cleaning: Dates – Postgres

- In addition to these functions, the SQL OVERLAPS operator on intervals is supported in most systems:
  - `(start1, end1) OVERLAPS (start2, end2)` is true if `start1` is earlier than `start2`, but `end1` is later than `start2` but earlier than `end2`, or vice-versa;
  - `(start1, length1) OVERLAPS (start2, length2)` is true as before, with `end1 = start1 + lenght1` and `end2 = start2 + length2`.

Dealing with Missing Data

- **Challenge:** Missing data appears in many forms
  $\Rightarrow$ no single standard representation
- **Common placeholders**:
  - -1 for numeric fields
  - empty string '' for categorical data

  $\Rightarrow$ should be converted to NULL

```
UPDATE Patients
SET Weight = NULL
WHERE Weight = -1;
```

- **Text markers**: 'n/a', 'N/A', 'NA'
  $\Rightarrow$ replace with NULL

Dealing with Missing Data (Cont.)

Key idea: `NULL` behaves like **unknown**, not a value.

Queries

What actually happens

```
SELECT *
FROM Patient
WHERE Weight > 220;
```

Rows with `NULL Weight` are `NOT returned.`

```
SELECT *
FROM Patient
WHERE Weight > 220 AND Height < 6;
```

If either condition is `NULL, the row is excluded.`

```
SELECT *
FROM Patient
WHERE Weight > 220 OR Weight <= 220;
```

Still misses rows where `Weight is NULL!`

Why? `NULL` $\Rightarrow$ comparison $=$ **UNKNOWN**

Example

Trying it out with dbfiddle:
https://www.db-fiddle.com/f/id2PgtbtT8NexwWxPv3xnQ/0

Dealing with Missing Data (Cont.)

- To identify NULL, use the special predicate IS [NOT] NULL:

```
SELECT COUNT(*)
FROM Patient
WHERE Weight IS NULL;
```

- As a rule, **aggregates ignore** NULLs, with some exceptions.
- Exception: COUNT(*) counts **all rows**, including those with NULLs.
- If **all** values in an attribute are NULL, COUNT returns 0.
- All other aggregates return NULL.
- For grouping, NULLs are treated as a group:

```
SELECT Weight, COUNT(*)
FROM Patient
GROUP BY Weight;
```

If Weight contains one or more NULLs,
a separate group will be created for them.

Example: NULLs in Practice

Sample data (Patient):

```
Name      Weight
----------------
Alice      210
Bob        NULL
Carol      250
David      NULL
```

Query 1:

```
SELECT COUNT(*)
FROM Patient
WHERE Weight IS NULL;
```

Result: 2

Query 2: COUNT vs COUNT(*)

```
SELECT COUNT(Weight), COUNT(*)
FROM Patient;
```

COUNT(Weight) = 2
COUNT(*) = 4

Query 3: GROUP BY

```
SELECT Weight, COUNT(*)
FROM Patient
GROUP BY Weight;
```

Output:

```
Weight   Count
--------------
210        1
250        1
NULL       2
```

Example

Trying it out with dbfiddle:
https://www.db-fiddle.com/f/dT1dAvuJ6xWqE2mtiDMEFs/0

Dealing with Missing Data (Cont.)

- Once records with missing values are identified, the general strategy is to **ignore (delete)** the affected data (either the attribute or the record), or to **predict** (also called **impute**) the absent values.
- **To delete incomplete records:**

```
DELETE FROM Table
WHERE attribute IS NULL;
```

- **To delete an attribute (column):**

```
ALTER TABLE Table
DROP COLUMN attribute;
```

Dealing with Missing Data (Cont.)

- **Imputing (predicting) missing values** may be done based on other values of the same attribute,
  or based on **correlated attributes**.
- **Substitute missing values (CASE):**

```
SELECT CASE WHEN attribute IS NULL THEN (newvalue)
            ELSE attribute END AS new_attribute
FROM Data;
```

- **Equivalent (simpler):**

```
SELECT COALESCE(attribute, newvalue) AS new_attribute
FROM Data;
```

COALESCE = first non-NULL value

Dealing with Missing Data (Cont.)

Trying this with dbfiddle:
`https://www.db-fiddle.com/f/4gX3AQFe8ZzXfNpc9mTcMo/0`

Dealing with Outliers

- An outlier is a value that is **not "normal"**, in the sense that it is **quite different** from other values in a domain.
- It tends to be **extreme** (for numerical attributes) and **unusual** (for categorical attributes).
- Detecting outliers is **tricky** because this is a **vague, context-dependent** concept: it is often unclear whether an outlier is an **error** or a **legitimate extreme value**.
- In some applications (e.g., **fraud detection**), outliers are **exactly what we are looking for!**

Dealing with Outliers (Cont.)

Outlier detection for single attributes depends on **the type of attribute**:

- For **nominal attributes**, we can calculate frequencies for each value; an outlier has **very low frequency**.
- For **numerical values**, it is **harder** to decide what is an outlier without assuming an **underlying distribution**.
- A common tactic is to find a distribution that **fits the data well**; values that **do not fit** can be considered outliers.
- With the **normal distribution**:
  about **95%** of values are within **2 std** of the mean[2],
  and over **99%** are within **3 std**,
  so values beyond that are **potential outliers**.

[2] **mean** $\pm 2 \times$ **(standard deviation)**

Dealing with Outliers (Cont.)

- However, in other distributions, for instance **exponential distributions** (or any **skewed / long-tail** distribution), it may **not be possible** to distinguish outliers from regular values using this test.
- Note that when **outliers are present**, they **influence the statistics** we use: they can **shift the mean** and **distort the standard deviation**.
- It may be a good idea to use **more robust statistics**, such as the **trimmed mean**.

Dealing with Outliers (Cont.)

- **Example setting:** Consider a dataset of people with an attribute **last (family) name**.
- **Potential issue:** Some names may be entered incorrectly (typos, misspellings, mispronunciations).
- **Key idea:** Errors are often **rare**, so they tend to appear as **low-frequency values**.
- **Strategy:** Focus on names that appear only once.
- **SQL query:**

```
SELECT last_name, COUNT(*) AS freq
FROM Dataset
GROUP BY last_name
HAVING COUNT(*) = 1;
```

- **Important:** This does **not confirm an error** — it only provides a **useful clue** for further checking.

Dealing with Outliers (Cont.)

- **Problem:** Using mean $\pm n\cdot$ std can be misleading when **extreme values already distort** the statistics.
- **Better idea:** Use **trimmed statistics** (remove extreme values first, then compute mean and std).
- **Step 1:** Ignore the **minimum** and **maximum** values.
- **Step 2:** Compute **trimmed mean** and **trimmed std**.
- **Step 3:** Flag values far from these trimmed statistics.
- **SQL implementation:**

```
WITH TrimmedStats AS
   (SELECT AVG(A) AS tmean, STDDEV(A) AS tdev
    FROM Data,
         (SELECT MAX(A) AS Amax FROM Data) AS T1,
         (SELECT MIN(A) AS Amin FROM Data) AS T2
    WHERE A < Amax AND A > Amin)
SELECT A
FROM Data, TrimmedStats
WHERE A < tmean - (2*tdev)
   OR A > tmean + (2*tdev);
```

- **Takeaway:** This method is **more robust** to extreme values.

Example

Trying this with dbfiddle:
https://www.db-fiddle.com/f/opCjWTF6Qro1gW9DWNFKcX/0

Dealing with Duplicates

- **What is the task?** We examine multiple rows and decide whether they refer to the **same real-world entity** (e.g., the same person, product, or event).
- **Key challenge:** Different rows may look slightly different (spelling, formatting, missing values), but still represent the **same object**.
- **Also called:** *deduplication, entity linkage, data linkage, record matching, entity resolution, co-reference, merge/purge*
- **Why is it hard?** We usually do **not have ground truth** to tell us which rows are truly duplicates.
- **Takeaway:** Duplicate detection is often **approximate and judgment-based**, not exact.

Dealing with Duplicates (Cont.)

- **Basic idea:** Choose a set of attributes that can act like a **primary key** (and possibly add a few more for safety).
- **Approach:** Check whether two rows have **exactly the same values** for these attributes.
- **Example:** In a people dataset, records with the same **name, address, and date of birth** may indicate duplicates.
- **SQL query:**

```
SELECT fname, lname, address, dob, COUNT(*)
FROM Data
GROUP BY fname, lname, address, dob
HAVING COUNT(*) > 1;
```

- **Limitation:** This method fails when:
  - values are **not exactly identical** (e.g., typos)
  - no attribute set can **uniquely identify** entities

Fuzzy Matching for Duplicate Detection

- **Challenge:** Categorical values are often not exactly identical (e.g., *Jones*, *Jonas*, *Jons*).
- **Goal:** Detect **similar values** that may refer to the **same entity**.
- **Idea:** Instead of exact equality, use **similarity measures**.
- **Two main approaches:**
  - **Sound-based** (how words sound)
  - **Edit-based** (how words differ)
- **Takeaway:** Use fuzzy matching when **exact matching fails**.

Fuzzy Matching: Sound-Based Similarity

- **Idea:** Compare strings based on how they **sound**.
- **How it works:**
  - Convert words to a **sound code** (SOUNDEX)
  - Compare the codes using `difference()`
- **Example:**

```
SELECT last_name
FROM Dataset
WHERE difference(last_name, 'Jones') > 3;
```

  0 = different 4 = very similar

- **Alternative:** `SOUNDEX(string1) = SOUNDEX(string2)`
- **Interpretation:** **Higher score ⇒ more similar**

Fuzzy Matching: Edit Distance

- **Idea:** Measure how many **edits** are needed to transform one string into another.
- **Allowed edits:**
  - insert delete replace
- **Example:**

```
SELECT last_name
FROM Dataset
WHERE levenshtein(last_name, 'Jones') < 2;
```

- **Interpretation:**
  - **0** → identical
  - small value → **very similar**
  - large value → different
- **Key point:** **Lower distance ⇒ more similar**

Fuzzy Matching: Numerical Examples

- **Same inputs, different measures:**

Name Pair	SOUNDEX	difference()	Levenshtein
Jones / Jonas	J520 / J520	4	1
Jones / John	J520 / J500	3	2
Jones / Smith	J520 / S530	0	5

- **How to read this:**
  - **SOUNDEX** → same code = similar sound
  - **difference()** → higher = more similar
  - **Levenshtein** → lower = more similar
- **Example:** *Jones vs Jonas*
  - Same sound → same code
  - difference = 4 → **very similar**
  - distance = 1 → **very similar**
- **Contrast:** *Jones vs Smith*
  - Different sound → different code
  - difference = 0 → **not similar**
  - distance = 5 → **far apart**

How Does SOUNDEX Work? (Intuition)

- **Goal:** Convert a word into a **code based on its sound**.
- **Basic steps:**
  - Keep the **first letter**
  - Replace remaining letters with **digits** based on sound groups
  - Remove **vowels and duplicates**
  - Keep only **4 characters** (**pad** with zeros for shorter strings and extra digits are **truncated** for longer strings)
- **Letter groups (same sound → same digit):**

1: B, F, P, V	4: L
2: C, G, J, K, Q, S, X, Z	5: M, N
3: D, T	6: R

- **Example: Jones**
  - J → keep **J**
  - O → ignored
  - N → 5, E → ignored, S → 2
  - Code: **J520**
- **Key idea:** Words that **sound alike** produce the **same code**.

How Does `difference()` Work? (Intuition)

- **Goal:** Measure how **similar two words sound**.
- **Key idea:** `difference()` does **not** compare strings directly. It compares their **SOUNDEX codes**.
- **How it works:**
  - Convert both strings using `SOUNDEX`
  - Compare the two codes character-by-character
  - **Count how many positions match**
- **Output:** A score between **0 and 4**
  - **4** $\rightarrow$ codes identical (very similar)
  - **0** $\rightarrow$ no similarity
- **Example:** Jones: `J520` Jonas: `J520` $\rightarrow$ `difference = 4`
- **Contrast:** Jones: `J520` and Smith: `S530` $\rightarrow$ `difference = 2`
- **Takeaway:** **Higher score $\Rightarrow$ more similar sound**

Example

Try `SOUNDS LIKE` with dbfiddle (for MySQL):
`https://www.db-fiddle.com/f/tmJR9BAeNoqMGtTBnQhcR4/0`

Try `SOUNDEX` with dbfiddle (for PostgreSQL):
`https://dbfiddle.uk/pHK4cRIT`

Compare `difference` and `levenshtein` with dbfiddle (for PostgreSQL):
`https://dbfiddle.uk/FJaI-dJV`

Dealing with Duplicates (Cont.)

- **Numerical attributes:** We can define similarity using **arithmetic distance**.
- **Idea:** Two values are **"close"** if their difference is small **relative to the scale of the data**.
- **Example (height):** Find individuals whose height is close to **180 cm**.

```
SELECT *
FROM Patients,
     (SELECT MAX(height) AS maxh FROM Patients) AS T
WHERE ABS(height - 180)/maxh < alpha;
-- alpha: small positive threshold (e.g., 0.05)
```

- **How it works:**
  - **ABS(height - 180)** → distance from 180
  - Divide by **max height** → normalize (percentage)
  - Compare with **alpha** → closeness threshold
- **Takeaway:** Numerical similarity is often defined as **relative difference**, not exact equality.

Dealing with Duplicates (Cont.)

- To determine whether two records are duplicates, we can apply different distances to different attributes and combine them:

```
SELECT P1.id AS id1, P2.id AS id2,
       P1.lname AS name1, P2.lname AS name2,
       P1.height AS height1, P2.height AS height2,
       P1.weight As weight1, P2.weight As weight2,
       levenshtein(P1.lname, P2.lname) / (maxl * 1.0)
         + ABS(P1.height - P2.height) / (hrange * 1.0)
         + ABS(P1.weight - P2.weight) / (wrange * 1.0)
       AS similarity
FROM Patients AS P1, Patients AS P2,
  (SELECT MAX(length(lname)) AS maxl FROM Patients) T1,
  (SELECT MAX(height)-MIN(height) AS hrange FROM Patients) T2,
  (SELECT MAX(weight)-MIN(weight) AS wrange FROM Patients) T3
WHERE P1.id < P2.id
ORDER BY similarity;
```

Example

Trying it out with dbfiddle: `https://dbfiddle.uk/imS3Kp7U`